

Rethinking the Black-Box Paradigm: Toward Structured Representations for Mechanistic Reasoning in Models of Code

Applicant: Kofi Sampong

Program: B.Eng., Computer Engineering

Email: ksampong@uoguelph.ca

Date: September 4, 2025

Abstract

Current practice in machine learning prioritizes scale and end-to-end training, yielding impressive performance but limited transparency. Post-hoc explanations rarely expose what models *actually* know, and—in tasks that require stepwise, causal reasoning—language models (LLMs) behave as fluent next-token predictors rather than mechanistic reasoners. Drawing on insights from interpretability (Molnar, 2020; Rudin, 2019), and on my recent work evaluating code embeddings, I propose a study that augments text models with **structured, program-aware representations** (graphs and traces) to encourage embeddings that encode *what code does* rather than *how it looks*. The project will investigate concept-bottleneck and contrastive objectives aligned to program structure (AST/CFG/DFG/SSA and execution traces), evaluate on correctness and clone robustness, and analyze the resulting latent spaces for causal order and compositionality.

1. Motivation & Problem Statement

LLMs have validated natural language as a surprisingly powerful representational medium (cf. Manning, 2019), but their success can create an illusion of understanding. In high-stakes or logically precise settings, three issues recur:

- **Opacity:** Black-box models make it hard to attribute decisions, verify invariants, or check causal pathways.
- **Post-hoc fragility:** Explanations often inherit training biases (Molnar, 2020) and can mislead when generalization fails.
- **Coherence $\neq$ Causality:** Fluent text does not imply a correct, step-wise mechanism; sequence models do not natively model goals, preconditions, or effects.

Claim. To elicit genuine *mechanistic* reasoning, we should **encode task structure explicitly**—causal relations, procedural order, and subgoal composition—so that model internals are shaped by program semantics rather than only surface statistics.

Pitfalls motivating this shift (tight summary): assuming generalization $\Rightarrow$ explanation quality; preferring complexity over sufficiency; ignoring feature dependencies; and conflating correlation with causation (Molnar, 2020; Rudin, 2019).

2. Context from Prior Work (Case Study)

As a research assistant on an educational tooling project for undergraduate C/C++, I implemented and evaluated three pipelines: **code summarization**, **code similarity**, and **code correctness classification**. Using CodeT5/CodeBERT fine-tuned on labeled samples from **Google Code Jam** and **CodeNet**, the correctness pipeline plateaued: models distinguished easy, Type-II refactors but failed on semantically diverse, Type-IV variants. Metrics (accuracy/F1) indicated pattern-matching rather than functional understanding. Careful error analysis confirmed that the models relied on shallow syntactic cues and token co-occurrence. In short, **fine-tuning black boxes did not induce mechanistic comprehension of program behavior**.

This experience suggests that, for logical tasks, *representation choice* may dominate model size: embeddings must be biased toward **state change, control flow, and data dependencies** if we expect robustness beyond superficial similarity.

3. Proposed Direction: Structured Representations for Mechanistic Understanding

Hypothesis. Treating code as *structured computation*—not just text—will yield embeddings that reflect causal order, compositionality, and semantics, improving correctness judgments and interpretability.

3.1 Representations

- **Multi-view program graphs:** Abstract Syntax Trees (AST), Control-Flow Graphs (CFG), Data-Flow Graphs (DFG), Static Single Assignment (SSA), and **execution traces** (instrumented or symbolic).
- **Primitive action vocabulary:** Start with ~10–20 primitives (assign, arithmetic op, compare, branch, loop, call/return, read/write, alloc/free, I/O). Larger behaviors are **compositions of**

subgraphs over these primitives.

3.2 Learning Objectives

- **Contrastive alignment:** Align raw code and its structured view (graph/trace) so paired views map nearby; encourage invariance under refactors/renames/dead code.
- **Concept bottlenecks:** Supervise interpretable *concept heads* that detect primitives and simple schemas (e.g., “counting loop,” “search then branch”).
- **Order/causality loss:** Penalize embeddings that violate known predecessor/successor or data-dependency constraints.

3.3 Evaluation

- **Correctness & robustness:** Held-out problems; Type-II/III/IV clones; adversarial refactors.
 - **Faithfulness:** Do concept activations correspond to ground-truth subgraphs/traces?
 - **Latent analysis:** PCA/t-SNE/UMAP to inspect cluster structure by primitive composition and control-flow motifs; probing for compositional generalization.
-

4. Scope, Early Goals & Milestones (coarse)

- **M1–M2:** Focused literature review; select representation toolkit; define v0 primitive vocabulary and schema library.
- **M2–M3:** Build a small, balanced dataset of paired (code, graph/trace) with labeled primitives; implement conversion/instrumentation pipeline.
- **M3–M4:** Train contrastive +working concept-bottleneck prototype; ablate representation views (AST-only vs AST+CFG+DFG vs trace).
- **M4–M5:** Evaluate on correctness and clone robustness; run faithfulness and invariance checks.
- **M6:** Synthesize results; identify scaling paths and publication venues; release dataset and code if feasible.

Deliverables (target): prototype model; paired dataset + converter; analysis report with visualizations; draft manuscript.

5. Anticipated Contributions & Impact

- **Representation:** A practical recipe for program-aware embeddings that encode causal order and composition.

- **Evaluation:** A robustness suite for correctness and semantic equivalence across refactors.
 - **Interpretability:** Concept heads and subgraph attributions that support *mechanistic* inspection rather than post-hoc heuristics.
-

6. Fit & Resources

This project aligns with labs working on program analysis, ML for code, interpretability, and graph representation learning. I seek mentorship on graph extraction, symbolic execution, and benchmarking. Compute needs are modest for prototypes (single-GPU training).

Author's Note on Supplementary Essays

While this document follows a conventional proposal structure, some of the motivation and reasoning are clearer in narrative form. I have therefore included three brief, standalone supplementary essays I have marked as—**Introduction**, **Context**, and **Proposed Methodology**—that expand on the ideas summarized here. A quick skim should provide a more faithful picture of the intent, scope, and trajectory than the compressed main text.

References (selected)

- Molnar, C. (2020). *Interpretable Machine Learning*. Leanpub.
- Rudin, C. (2019). “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead.” *Nature Machine Intelligence*.
- Manning, C. D. (2025). Remarks on language as a representation for knowledge [talk].
-
- Allister Pais, N., et al. (2025). *Evaluating Source Code Embeddings for Educational Downstream Tasks*. IEEE UEMCON 2025, status: under review.
- (Additional code-representation and graph-learning citations to be added in the full draft.)
-